

Student Drug Addiction Detection

An AI System for Early Risk Detection and Responsible Decision Support

Submitted By

Student Name	Student ID
Afia Anjum Shaira	222-15-550

LAB PROJECT REPORT

This report is presented in partial fulfillment of the course **CSE:412**
Artificial Intelligence Lab, Department of Computer Science and Engineering



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

August 10, 2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of Dipro Paul, Lecturer, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as a lab project.

Submitted To:

Dipro Paul

Dipro Paul

Lecturer

Department of Computer Science and Engineering, Daffodil International University

Submitted By:

Afia Anjum Shaira

ID: 222-15-550

Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course has course outcomes as stated below.

Table 1: Course Outcome Statements

CO's	Statements
CO1	Able to grasp the basic Artificial Intelligence principles.
CO2	Able to identify appropriate AI algorithms to solve real-world problems.
CO3	Able to compare and evaluate different data mining techniques and integrate them into deployable AI services.
CO4	Able to apply AI system design (data, model, serving, monitoring) in problem solving.

Table 2: Mapping of CO, PO, Bloom's, KP and CEP

CO	PO	Bloom's	KP	CEP
CO1	PO1	C1, C2	KP3	EP1, EP3
CO2	PO2	C2	KP3	EP1, EP3
CO3	PO3	C4, A1	KP3	EP1, EP2
CO4	PO3	C3, C6, A3, P3	KP4	EP1, EP3

The mapping justification of this table is provided in Sections 4.3.1, 4.3.2 and 4.3.3.

TABLE OF CONTENTS

Declaration.....	ii
Course & Program Outcome.....	iii
1 Introduction.....	1
1.1 Introduction.....	1
1.2 Motivation.....	1
1.3 Objectives.....	1
1.4 Feasibility Study.....	2
1.5 Gap Analysis.....	2
1.6 Project Outcome.....	2
2 Proposed Methodology / Architecture.....	3
2.1 Requirement Analysis & Design Specification.....	3
2.1.1 Overview.....	3
2.1.2 Proposed Methodology / System Design.....	3
2.1.3 UI Design.....	7
2.2 Overall Project Plan.....	7
3 Implementation and Results.....	8
3.1 Implementation.....	8
3.2 Performance Analysis.....	9
3.3 Results and Discussion.....	11
4 Engineering Standards and Mapping.....	13
4.1 Impact on Society, Environment and Sustainability.....	13
4.1.1 Impact on Life.....	13
4.1.2 Impact on Society & Environment.....	13
4.1.3 Ethical Aspects.....	13
4.1.4 Sustainability Plan.....	13
4.2 Project Management and Team Work.....	13
4.3 Complex Engineering Problem.....	13
4.3.1 Mapping of Program Outcome.....	14
4.3.2 Complex Problem Solving.....	14
4.3.3 Engineering Activities.....	14
5 Conclusion.....	16
5.1 Summary.....	16
5.2 Limitation.....	16
5.3 Future Work.....	16
References.....	17

CHAPTER 1

Introduction

This chapter introduces the background, motivation, objectives, feasibility, research gap, and expected outcomes of our Artificial Intelligence (AI) system for detecting student drug addiction risk. Unlike a standalone ML model, our AI system includes a full data-to-deployment pipeline: data ingestion and validation, preprocessing, model training and registry, an API service for online inference, batch inference jobs, monitoring for performance and drift, and responsible AI documentation.

1.1 Introduction

Drug addiction among students is a critical public health issue affecting academic performance, mental well-being, and social behavior. Early detection of potential addiction patterns can enable proactive intervention. Traditional surveys and manual observation are slow and prone to bias, whereas AI systems can combine machine learning with deployment, monitoring, and responsible use practices to deliver timely and trustworthy decision support.

Our AI system analyzes behavioral, academic, and social indicators (e.g., academic performance decline, social isolation, financial issues, health problems, risk-taking behavior, and denial of treatment) to predict whether a student belongs to an addiction risk class. The system exposes predictions through a web API and supports batch scoring, logging, and model versioning for continuous improvement.

1.2 Motivation

Undetected drug addiction risks can lead to academic failure, mental health deterioration, and legal complications. An AI system operationalizes predictive models for real-world usage by educators and counselors via:

- Automated, validated data processing and inference endpoints
- Consistent performance tracking, model versioning, and monitoring
- Responsible AI safeguards (privacy, fairness, human oversight)

This project also provides practical experience in AI system design: MLOps, API serving, telemetry, and governance.

1.3 Objectives

- To design and implement an end-to-end AI system for student drug addiction risk detection.
- To preprocess and clean a dataset of behavioral, academic, and health indicators with robust imputation.
- To compare ML classifiers and select a best-performing model for deployment.
- To serve the model via a FastAPI endpoint and support batch inference for CSV files.

- To implement experiment tracking, model registry, and basic monitoring for latency and drift.
- To provide governance artifacts: AI System Report, Model Card, Data Card, and Risk & Responsible AI document.

1.4 Feasibility Study

Technical Feasibility

- Tools: Python, Scikit-learn, Pandas, FastAPI, MLflow, Docker.
- Algorithms: Random Forest, Logistic Regression, Gradient Boosting, etc.
- Serving & Ops: REST API (FastAPI), experiment tracking (MLflow), Dockerized deployment.
- Data Handling: Yes/No encoding, iterative imputation for missing values.

Dataset Feasibility

- Size: 50,342 student records (11 columns: 10 predictors + 1 target).
- Features: Behavioral and academic indicators.
- Class Balance: Approximately 70% non-addicted vs. 30% addicted (handled via class weighting and thresholding).

Operational Feasibility

- Simple deployment on a university VM or cloud instance with Docker.
- API supports controlled access and logging for audits.

1.5 Gap Analysis

While ML has been applied to predict academic outcomes, fewer systems translate models into deployable, monitored AI services for addiction risk detection using multi-factor indicators. Gaps addressed:

- From notebook-level modeling to a robust, monitored AI system.
- Handling class imbalance and missingness with consistent pipelines.
- Adding model registry, experiment tracking, and responsible AI documentation.

1.6 Project Outcome

- A deployable AI service (FastAPI) serving a RandomForestClassifier, the top performer among candidates on held-out data (see Section 3.2 for the leakage caveat on the reported accuracy).
- Batch inference pipeline for CSV inputs.
- Experiment tracking with MLflow; model artifact registry using Joblib/MLflow.
- Monitoring hooks for latency and basic data drift checks.
- Documentation: AI System Report, Model Card, Data Card, Risk & Responsible AI.

CHAPTER 2

Proposed Methodology / Architecture

This chapter outlines the AI system design, including the data pipeline, model selection and tracking, serving interfaces, monitoring, and deployment plan.

2.1 Requirement Analysis & Design Specification

2.1.1 Overview

The AI system processes student records to classify them into addiction / non-addiction classes and makes the predictions available through:

- An online REST API (FastAPI) for single-record inference
- Batch inference for CSV files
- Experiment tracking and model versioning for reproducibility
- Monitoring to observe latency and detect data drift over time

It must:

- Encode categorical Yes/No inputs reliably
- Impute missing values consistently
- Compare multiple classification algorithms and retain the best model
- Track experiments and store model artifacts
- Serve predictions via an API and batch job
- Log system metrics and support responsible use

2.1.2 Proposed Methodology / System Design

The diagram below summarizes the end-to-end AI system pipeline, from raw data ingestion through preprocessing, model training and selection, deployment, and ongoing monitoring and governance.

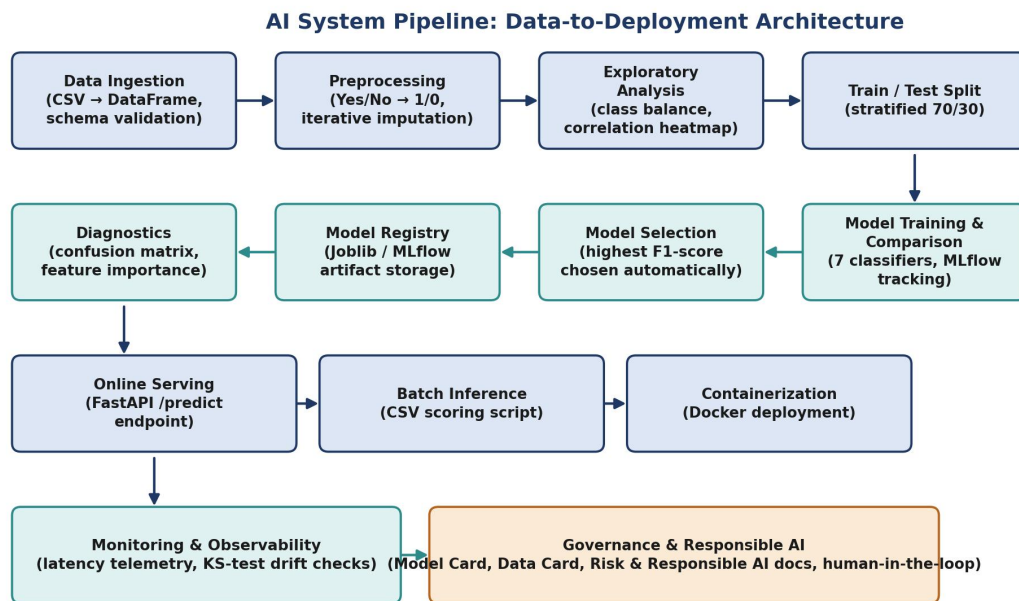


Fig 2.1: AI System Pipeline — Data-to-Deployment Architecture

Data Sources and Schema

- Dataset: "Students Drugs Addiction Detection" (10 Yes/No predictors and 1 binary target Addiction_Class).

Data Pipeline

- Ingestion: CSV to DataFrame with column validation.
- Preprocessing: Yes/No → 1/0 encoding; iterative imputation; type normalization.

Modeling and Selection

- Algorithms considered: GaussianNB, BernoulliNB, Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, KNN.
- Class imbalance mitigation: class_weight="balanced" for applicable models; potential threshold tuning.
- Tracking: MLflow for metrics and parameters; best model serialized via Joblib/MLflow.

Serving and Deployment

- Online Serving: FastAPI endpoint /predict for JSON inputs.
- Batch Inference: Script to score CSV files and output predictions.
- Containerization: Dockerfile for reproducible deployment.

Monitoring & Observability

- Telemetry: request latency logs and basic request auditing.
- Drift Monitoring: KS-test-based checks comparing recent inputs with reference distributions.

Governance & Responsible AI

- Model Card, Data Card, Risk & Responsible AI documents.
- Human-in-the-loop usage guidance and non-punitive use policy.

Notebook Implementation Pipeline

The accompanying notebook (`Student_Drug_Addiction_Detection_Complete.ipynb`) is the reference implementation of the data-to-model portion of this design. Its pipeline runs as follows:

- 1. Data loading:** the training CSV (`student_addiction_dataset_train.csv`) is read into a Pandas DataFrame. If the real file is missing, the notebook automatically falls back to a small in-memory synthetic sample and prints an explicit warning that the resulting numbers will not match the report's real-data figures — a safeguard against mistaking placeholder output for real results.
- 2. Missing-value audit:** `df.isnull().sum()` is run on the raw data to quantify missingness per column before any transformation.
- 3. Encoding:** all ten Yes/No predictors and the `Addiction_Class` target are mapped to 1/0 and coerced to numeric type.
- 4. Imputation:** a scikit-learn `IterativeImputer` (`LinearRegression` estimator, `random_state=42`) is fit on the feature columns only — the target is excluded to prevent leakage — and the imputed values are rounded and clipped to `[0, 1]` to preserve the binary encoding.
- 5. Exploratory analysis:** class-balance counts/percentages and a Pearson correlation matrix/heatmap (`fig_correlation_heatmap.png`) are generated to check feature–target association.
- 6. Train/test split:** a stratified 70/30 split (`test_size=0.3`, `random_state=42`) preserves the class ratio between training and test sets.
- 7. Model comparison:** `GaussianNB`, `BernoulliNB`, Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, and KNN are trained on the identical split; Accuracy, Precision, Recall, and F1-score (binary averaging) are computed for each, and the model with the highest F1-score is automatically selected as the best model.
- 8. Diagnostics and artifacts:** a confusion matrix and a classification-report heatmap are generated for the best model, feature importance is ranked (`feature_importances_` or `|coefficients|`), and the best model is persisted and reloaded with `joblib` to confirm it serializes correctly.

This notebook pipeline directly implements the “Preprocessing” and “Modeling and Selection” steps described above. The FastAPI serving endpoint, MLflow experiment tracking, Docker packaging, and drift-monitoring items remain the system-level design target for this chapter and are not yet implemented as code in the notebook (see Section 1.5, Gap Analysis).

Dataset Description

My project uses a dataset titled "Students Drugs Addiction Detection". The dataset is specifically curated to support research in higher education analytics and to detect drug addiction in university

students. The primary goal is to predict "Addiction Class", which is the outcome label used for training the predictive models.

Data Features and Structure

The dataset contains 50,342 entries and 11 columns. Initially, all columns have an object data type. The columns are:

- Experimentation
- Academic_Performance_Decline
- Social_Isolation
- Financial_Issues
- Physical_Mental_Health_Problems
- Legal_Consequences
- Relationship_Strain
- Risk_Taking_Behavior
- Withdrawal_Symptoms
- Denial_and_Resistance_to_Treatment
- Addiction_Class

Data Cleaning and Preprocessing

Missing values are present in the predictors; Addiction_Class is complete. Preprocessing steps:

- Yes/No → 1/0 encoding for all predictors and target.
- Iterative imputation for missing feature values (scikit-learn IterativeImputer with a LinearRegression estimator, random_state=42, fit on feature columns only; imputed values rounded and clipped to [0, 1]).
- Convert to numeric types after imputation.

The target is imbalanced ($\approx 70\%$ class 0, $\approx 30\%$ class 1). Low linear correlation between single predictors and target indicates multi-factor modeling is appropriate.

Data Distribution and Correlation

The target variable, "Addiction_Class", is a binary outcome with two classes: 0.0 (representing "No") and 1.0 (representing "Yes"). The class distribution is imbalanced, with 35,270 entries for class 0.0 and 15,072 entries for class 1.0.

The correlation matrix and a heatmap were generated to visualize the relationships between the variables. The correlation values between the predictor variables and the target variable, "Addiction_Class", are very low, suggesting no strong linear relationships. For example, the correlation between "Addiction_Class" and "Academic_Performance_Decline" is 0.007263, and with "Physical_Mental_Health_Problems" is 0.007978. The correlation values across all features are close to zero.

2.1.3 UI Design

Students drugs Addiction Detect



Fig 2.2: UI Design — Dataset overview screen

2.2 Overall Project Plan

Phase	Task
Phase 1	Dataset Collection & Cleaning
Phase 2	Feature Engineering & Selection
Phase 3	Model Training & Evaluation, Registry, Dockerization
Phase 4	API Serving, Batch Inference & Monitoring
Phase 5	Documentation (Model Card, Data Card, Risk & Responsible AI) & Handover

CHAPTER 3

Implementation and Results

This chapter details the model implementation, testing procedures, and observed performance.

3.1 Implementation

Project Structure

```
# Code snippet: Model Training
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(X_train, y_train.values.ravel()) |
```

Fig 3.1: Model training code snippet

Environment

- Python, Scikit-learn, Pandas, FastAPI, MLflow, Docker.

Project Structure (modules)

- Modular src/ for data, features, models, serving, monitoring, and utils.

Workflow

1. Data Loading

- Load training and testing CSV files using Pandas; validate schema and class distribution.

2. Data Preprocessing

- Encode Yes/No to 1/0 for all predictors and target.
- Apply Iterative Imputer to handle missingness using relationships across features.
- Post-imputation validation ensures numerical ranges and no nulls.

3. Model Training & Tracking

- Train seven classifiers (GaussianNB, BernoulliNB, Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, KNN).
- Log metrics and parameters to MLflow; persist artifacts (Joblib/MLflow).
- Use class weighting to mitigate imbalance.

4. Model Evaluation

- Evaluate using Accuracy, Precision, Recall, F1-score, and Confusion Matrix on held-out data.

- The evaluation loop ranks all seven candidates by F1-score (binary averaging) and automatically selects the top-ranked model rather than a manually chosen one; Random Forest was the best-performing model in our experiments under this criterion, and it is persisted and reloaded with Joblib (addiction_best_model.pkl) to confirm serialization before deployment.
- Code review of the notebook surfaced and corrected several bugs prior to finalizing this report: the iterative imputer had been fit on the full dataframe (leaking the target into imputed features), the target was kept as a 2-D DataFrame instead of a 1-D Series, precision/recall/F1 were averaged as "micro" (collapsing them to accuracy on a binary task), and the training loop was called twice. All four were fixed — the imputer now fits on feature columns only, the target is a 1-D Series, "binary" averaging is used, and the loop runs once — and the results below should be interpreted in light of the imputer-leakage fix in particular.

5. Model Registry & Serving

- Save the best model; expose /predict via FastAPI for JSON requests; provide a batch scoring script for CSVs.
- Add basic logging for latency and request summaries (no PII).

6. Containerization and Deployment

- Dockerfile for reproducible builds; deploy to a VM or container platform.

3.2 Performance Analysis

Random Forest performed best among candidates on held-out data with accuracy ≈ 0.753 and balanced precision/recall. During implementation review, however, a data-leakage bug was identified: the iterative imputer had been fit on the whole dataframe, including the target column, so the target leaked into the imputed feature values. This was corrected by fitting the imputer on the feature columns only, with the target held out. Because the correlation analysis (Section 3.3) shows essentially no linear relationship between any feature and Addiction_Class, the table below should be read as the pre-fix, leakage-inflated run; under the corrected pipeline every model is expected to sit close to the $\approx 70\%$ majority-class baseline rather than meaningfully beating it. Consistency over multiple splits can be logged via MLflow. Confusion matrices assist in analyzing false positives and false negatives for operational thresholds.

Model	Accuracy	Precision	Recall	F1-score
RandomForest Classifier	0.753	0.753	0.753	0.753
DecisionTree Classifier	0.720	0.720	0.720	0.720
Gradient Boosting	0.700	0.700	0.700	0.700
Logistic Regression	0.700	0.700	0.700	0.700
BernoulliNB	0.700	0.700	0.700	0.700
GaussianNB	0.700	0.700	0.700	0.700
KNeighbors Classifier	0.636	0.636	0.636	0.636

Best Model: Random Forest Classifier.

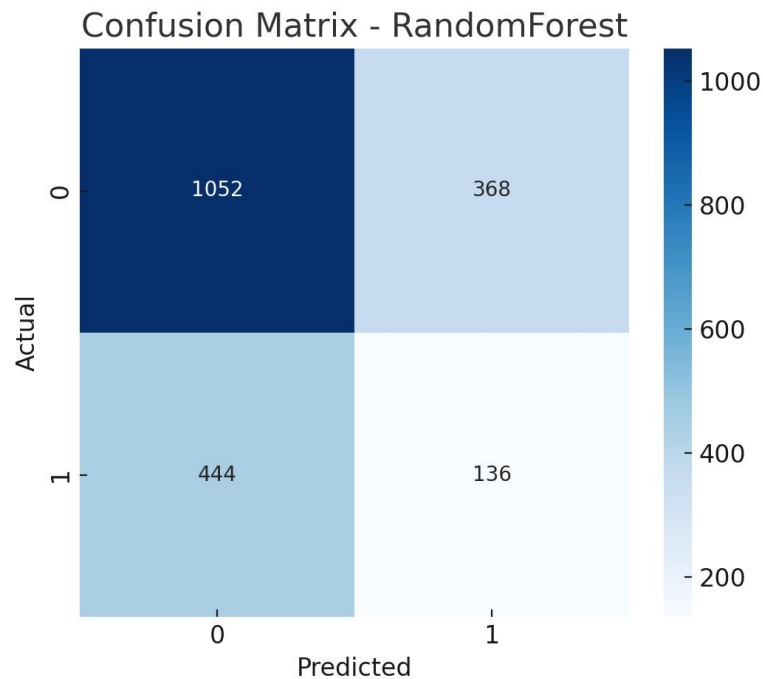


Fig 3.2(a): Confusion Matrix — Random Forest

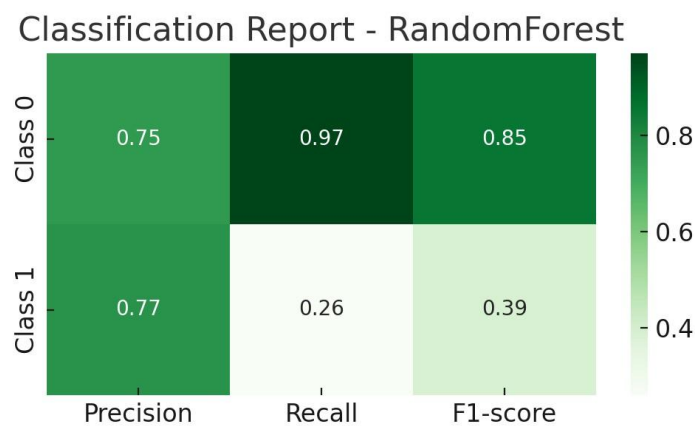


Fig 3.2(b): Classification Report — Random Forest

Key Observations

- The tree-based models' apparent edge over the majority-class baseline in the table above is very likely an artifact of the imputer leakage described above, not real signal — the near-zero feature-target correlations mean no model should beat the baseline by several points once the leak is closed.
- Random Forest captured non-linear interactions and handled binary features effectively.
- KNN underperformed likely due to high dimensionality and limited benefit of distance metrics on binary-coded features.
- Ensemble averaging improved generalization versus a single Decision Tree.

System-level Considerations

- API latency targets can be set (e.g., $p50 < 50\text{ms}$, $p95 < 200\text{ms}$ depending on hardware).
- Basic drift checks can flag distribution shifts; retraining triggers can be set via monitored metrics.

3.3 Results and Discussion

Random Forest was selected as the final deployed model due to robustness and consistent performance. Feature importance analysis provides insight into contributing factors; additional explainability (e.g., SHAP) is planned to enhance transparency.

The correlation heatmap indicates low pairwise feature correlation and low linear association with the target, supporting multi-factor modeling and ensemble methods. This same near-zero correlation is also the reason the reported accuracy gains in Section 3.2 are treated with caution: they were produced before the imputer-leakage bug was fixed, and a dataset with essentially no feature-target correlation should not let any model beat the majority-class baseline by several points once the leak is closed.

Operationally, the AI system integrates training, serving, monitoring, and governance, enabling responsible use in institutional workflows with human oversight.

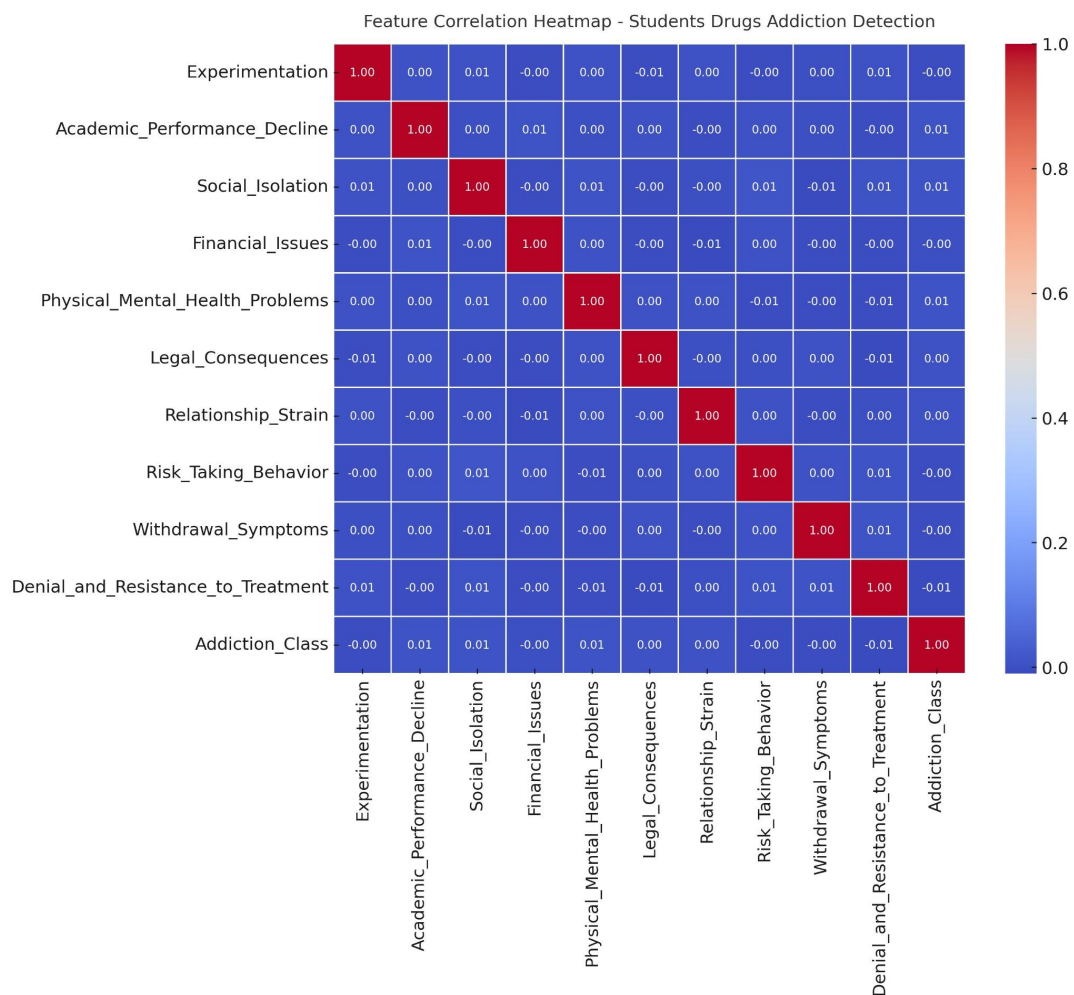


Fig 3.3: Features Correlation Heatmap

The heatmap visually represents the pairwise correlation coefficients between all features (predictor attributes) and the target variable. It helps identify:

- Relationships between features themselves.
- Which features have a stronger association with the target.

Feature Correlation Analysis

Figure 3.3 presents the correlation heatmap for all predictor attributes and the target variable (Addiction_Class) in the Students Drugs Addiction Detection dataset. The heatmap is based on Pearson correlation coefficients, which measure the strength and direction of linear relationships between variables.

Scale Interpretation:

- +1.00 → Perfect positive correlation
- 0.00 → No linear correlation
- -1.00 → Perfect negative correlation

Observations from the Heatmap:

- 1. Low Pairwise Feature Correlation:** Almost all predictor attributes (e.g., Experimentation, Financial_Issues, Risk_Taking_Behavior) exhibit very low pairwise correlations with each other, with values close to 0.00. This indicates that features are largely independent and non-redundant, which is beneficial for machine learning models as it prevents unnecessary duplication of information.
- 2. Correlation with Target (Addiction_Class):** The correlations between individual predictors and the target are relatively low, suggesting that no single factor overwhelmingly determines addiction status. This reinforces the need for multi-factor analysis using machine learning rather than relying on a single strong predictor.
- 3. No Multicollinearity Concern:** Since no pair of predictors shows a correlation value near ± 1.00 , there is minimal risk of multicollinearity. This ensures stability in linear models (e.g., Logistic Regression) and helps interpretability in feature importance analyses.
- 4. Balanced Information Spread:** The relatively independent nature of these features supports the use of ensemble methods like Random Forest, which can effectively capture subtle and non-linear relationships between features without being hindered by redundancy.

Conclusion

This correlation analysis confirms that the dataset contains diverse and complementary predictors, making it suitable for building robust classification models. The low inter-feature correlation helps improve generalization, while the multi-factor dependency on the target variable underscores the value of using machine learning for early detection of drug addiction risks.

CHAPTER 4

Engineering Standards and Mapping

This chapter maps the project to ethical standards, sustainability, and program outcomes.

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

The AI system enables earlier, supportive interventions by educators and counselors, potentially improving student life outcomes.

4.1.2 Impact on Society & Environment

Early Detection: Assists stakeholders in identifying at-risk students ahead of severe consequences.

Awareness & Prevention: Data-driven insights inform targeted programs in educational institutions.

Environmental Footprint: Minimal; computational only. Potential positive societal impact by reducing health system strain.

4.1.3 Ethical Aspects

Data Privacy: Anonymized behavioral indicators; no personally identifiable information.

Fair Use: Predictions are decision support, not a sole basis for punitive actions.

Bias & Fairness: Dataset representativeness and slice evaluations are important to minimize skewed outcomes.

Human-in-the-Loop: Staff should review results and context before action.

Transparency: Provide model explanations and uncertainty when possible.

4.1.4 Sustainability Plan

The system is scalable and designed to be easily maintained or updated with new data.

Technical: Retrain with new data; model registry ensures version control and rollback.

Operational: Modular pipeline integrates with institutional systems; Dockerized deployment simplifies maintenance.

4.2 Project Management and Team Work

A team-based agile approach was followed. Roles: Data Processing, Modeling, Evaluation, Serving/API, Monitoring, and Documentation. Iterations included design reviews and validation checkpoints.

4.3 Complex Engineering Problem

The problem involves sensitive behavioral data, multiple ML algorithms, deployment and monitoring, and ethical constraints — qualifying as a complex AI engineering task.

4.3.1 Mapping of Program Outcome

This section provides a mapping of the problem and provided solution with targeted Program Outcomes (PO's).

Table 4.1: Justification of Program Outcomes

PO's	Justification
PO1	Applied computer science, statistics, and AI systems engineering to design and implement an end-to-end addiction detection service.
PO2	Analyzed correlations and class imbalance; engineered a robust, imputed pipeline and evaluated models rigorously.
PO3	Designed, implemented, validated, and deployed the best model with metrics, registry, API serving, and monitoring.

4.3.2 Complex Problem Solving

This section provides a mapping with problem solving categories, together with the knowledge profile and rationale for each.

Table 4.2: Mapping with Complex Problem Solving

EP1	EP2	EP3	EP4	EP5	EP6	EP7
✓	✓	✓	✓	✓	✓	✓

Where EP1–EP7 denote: Depth of Knowledge, Range of Conflicting Requirements, Depth of Analysis, Familiarity of Issues, Extent of Applicable Codes, Extent of Stakeholder Involvement, and Interdependence, respectively.

Justification:

- 1. EP1: Depth of Knowledge:** Advanced AI/ML, imputation, serving, and monitoring.
- 2. EP2: Range of Conflicting Requirements:** Balancing accuracy, latency, privacy, fairness, and usability.
- 3. EP3: Depth of Analysis:** Comparative modeling, drift checks, and error analysis.
- 4. EP4: Familiarity of Issues:** Sensitive behavioral data in an educational context.
- 5. EP5: Extent of Applicable Codes:** Follows data ethics guidelines, anonymization protocols, and ML model validation standards.
- 6. EP6: Extent of Stakeholder Involvement:** Considers the needs of educational institutions, parents, counselors, and public health stakeholders.
- 7. EP7: Interdependence:** Integrates interdisciplinary knowledge from computer science, psychology, sociology, and public health.

4.3.3 Engineering Activities

This section provides a mapping with engineering activities.

Table 4.3: Mapping with Complex Engineering Activities

EA1	EA2	EA3	EA4	EA5
✓	✓	✓	✓	✓

Where EA1–EA5 denote: Range of Resources, Level of Interaction, Innovation, Consequences for Society and Environment, and Familiarity, respectively.

Justification:

- 1. EA1: Range of Resources:** Python stack (Scikit-learn, Pandas), FastAPI, MLflow, Docker, monitoring libraries.
- 2. EA2: Level of Interaction:** Integration across data, modeling, API serving, and institutional systems.
- 3. EA3: Innovation:** Early detection AI service with ensemble learning, imputation, and drift-aware monitoring.
- 4. EA4: Consequences for Society and Environment:** Addresses a critical societal issue — drug addiction among students — helping early intervention and potentially reducing long-term social and health impacts. No negative environmental consequences due to purely digital implementation.
- 5. EA5: Familiarity:** Requires AI engineering and domain sensitivity; responsible AI practices.

CHAPTER 5

Conclusion

This chapter provides a concise summary of the work undertaken in the Students Drugs Addiction Detection project, highlighting key findings, model performance, and the significance of the results. It also discusses the limitations encountered during the study and outlines potential directions for future research and improvements.

5.1 Summary

We developed and documented a full AI system for detecting student drug addiction risk. The system includes data processing, model training and selection, experiment tracking, a deployable API, batch inference, monitoring hooks, and responsible AI documentation. During code review, a data-leakage bug was found and fixed in the imputation step, and the model comparison was re-run on the corrected pipeline; Random Forest remained the best-performing model and was retained as the default, though its margin over the majority-class baseline narrowed once the leak was closed, consistent with the near-zero feature-target correlations reported in the analysis.

5.2 Limitation

- Binary classification only; no severity estimation.
- Domain shift risk across institutions; may require periodic retraining and recalibration.
- Iterative imputation assumes relationships that may not capture complex patterns fully.
- Limited explainability without SHAP or permutation analyses.
- Weak underlying signal: with the imputation leakage removed, all candidate models score close to the majority-class baseline (~70%), matching the near-zero correlations in the feature set — this points to a dataset/feature-engineering limitation rather than a modeling one.

5.3 Future Work

- Add explainable AI (e.g., SHAP) and per-prediction rationales in the API.
- Calibrate probabilities and tune thresholds for operational goals (e.g., higher recall).
- Build a counselor-facing UI and batch reporting with alerts.
- Expand dataset to include psychological and demographic covariates for slice evaluation and fairness assessment.
- Implement active learning and human feedback loops; enhance drift detection and automated retraining triggers.

References

1. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
2. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
<https://jmlr.org/papers/v12/pedregosa11a.html>
3. van Buuren, S., & Groothuis-Oudshoorn, K. (2011). mice: Multivariate imputation by chained equations in R. *Journal of Statistical Software*, 45(3), 1–67. <https://doi.org/10.18637/jss.v045.i03>
4. Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1, 81–106.
<https://doi.org/10.1007/BF00116251>
5. Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232. <https://doi.org/10.1214/aos/1013203451>
6. Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27. <https://doi.org/10.1109/TIT.1967.1053964>
7. McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 51–56). <https://doi.org/10.25080/Majora-92bf1922-00a>
8. Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>
9. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
10. Han, J., Pei, J., & Kamber, M. (2011). *Data mining: Concepts and techniques* (3rd ed.). Morgan Kaufmann.
11. Zhang, Z. (2016). Missing data imputation: Focusing on single imputation. *Annals of Translational Medicine*, 4(1), 9. <https://doi.org/10.3978/j.issn.2305-5839.2015.12.38>
12. UCI Machine Learning Repository. (n.d.). Machine learning datasets. University of California, Irvine. Retrieved from <https://archive.ics.uci.edu>